

PRESENTATION PROBLEMS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Presentation 1

In the card game *Alchemy: The Assembling*, you are given a hand B of n monster cards, organized in a row. Each card has an integer level B_i . Fusing two adjacent cards together creates a new card, whose level is equal to the *product* of the original levels. This also incurs a mana cost equal to the level of the newly-created card.

Your objective is to fuse all the cards in your hand into one single, ultimate card, using *as little mana as possible*.

Develop an efficient algorithm to find the optimal play for a given hand B of length n .

Example

Input: $B = [5, 2, 3, 4]$

Output: optimal play is:

1. Merge adjacent (5, 2) in $[5, 2, 3, 4]$
 - Cost: $5 \times 2 = 10$
 - After merge: $[10, 3, 4]$
2. Merge adjacent (3, 4) in $[10, 3, 4]$
 - Cost: $3 \times 4 = 12$
 - After merge: $[10, 12]$
3. Merge adjacent (10, 12) in $[10, 12]$
 - Cost: $10 \times 12 = 120$
 - After merge: $[120]$

The total mana cost incurred by this sequence is:

$$10 + 12 + 120 = 142$$

which is the minimum achievable.

Presentation 1 Solution

1: The Last Decision

The last decision occurs when our hand has only 3 cards. We will then decide whether to merge B_1 and B_2 , or B_2 and B_3 , in order to minimize the total cost.

2: Subproblem Definition

$p[i, j]$: the minimum cost achievable in merging the subarray $B[i...j]$.

$m[i, j]$: the resulting value of $B[i...j]$ after all the cards in have been merged.

$r[i, j]$: the index where the last merge takes place in order to achieve the minimum cost for the subarray $B[i...j]$.

3: Recurrence Relation

$$p[i, j] = \begin{cases} NIL & \text{if } i > j \\ 0 & \text{if } i = j \\ \min_{i \leq k < j} (p[i, k] + p[k + 1, j] + m[i, k] \times m[k + 1, j]) & \text{otherwise} \end{cases} \quad (1)$$

$$m[i, j] = \begin{cases} NIL & \text{if } i > j \\ B[i] & \text{if } i = j \\ m[i, k] \times m[k + 1, j] & \text{otherwise} \end{cases} \quad (2)$$

$$r[i, j] = \begin{cases} NIL & \text{if } i > j \\ 0 & \text{if } i = j \\ \operatorname{argmin}_{i \leq k < j} (p[i, k] + p[k + 1, j] + m[i, k] \times m[k + 1, j]) & \text{otherwise} \end{cases} \quad (3)$$

4: Pseudocode

Algorithm 1 Compute Minimum Cost

```

1: function MINCOST( $B$ )
2:    $n = \text{len}(B)$ 
3:    $p \leftarrow n \times n$  array with 0 in the diagonal
4:    $m \leftarrow n \times n$  array with card levels in the diagonal
5:   for  $l \leftarrow 2$  to  $n + 1$  do
6:     for  $i \leftarrow 1$  to  $n - l + 2$  do
7:        $j \leftarrow l + i - 1$ 
8:       for  $k \leftarrow i$  to  $j$  do
9:         Compute  $p[i, j]$  and  $p[i, j]$  using the recurrence relation
10:        Record merge in  $m[i, j]$ 
11:      end for
12:    end for
13:  end for
14:  return  $p[0, n]$ 
15: end function

```

Algorithm 2 Reconstruct Merge Sequence

```

1: function RECONSTRUCT( $i, j$ )
2:   if  $i = j$  then
3:     return  $B[i]$ 
4:   end if
5:    $k \leftarrow r[i, j]$ 
6:    $leftMerge \leftarrow \text{RECONSTRUCT}(i, k)$ 
7:    $rightMerge \leftarrow \text{RECONSTRUCT}(k + 1, j)$ 
8:   return “( $leftMerge$   $rightMerge$ )”
9: end function

```

5: Example

$B = [5, 2, 3, 4]$

		i/j	1	2	3	4
$p[i, j] =$	1		0	10	36	142
	2		–	0	6	30
	3		–	–	0	12
	4		–	–	–	0
		i/j	1	2	3	4
$m[i, j] =$	1		5	10	30	120
	2		–	2	6	24
	3		–	–	3	12
	4		–	–	–	4

$$r[i, j] = \begin{array}{c|cccc} i/j & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 1 & 1 & 2 \\ 2 & - & 0 & 2 & 3 \\ 3 & - & - & 0 & 3 \\ 4 & - & - & - & 0 \end{array}$$

Reconstructed solution: ((5 2) (3 4))

6: Proof of Optimal Substructure

The subproblems for this problem are independent of each other, because the total mana cost of merging all the cards in a subarray $B[i...j]$ depends only on the order in which the cards were merged- not on how the single card produced after all are merged, is merged with any cards outside the subarray.

Given: Let S be an optimal solution to the subproblem covering the entire array B . That solution will fuse two cards x and y , where x has been created by fusing all cards $B[1...c]$ and y has been created by fusing $B[c + 1...n]$. We will call the process by which $B[1...c]$ were fused together to create x , the subproblem S' .

Claim: S' is an optimal solution to the subproblem of the cards $B[1...c]$.

Proof by Exchange of Arguments: Assume that our claim is **false**. This means that S' is **not** the optimal solution to the subproblem of the cards $B[1...c]$. Then, there must be an alternative optimal solution S'' that is feasible and provides a lower mana cost than S' . Using S'' , we can reconstruct the optimal solution for the original problem in the following way:

$$S^* = S'' + B[x] \times B[y] < S' + B[x] \times B[y] = S$$

Here, S^* is a feasible solution for the original problem because it merges the same two adjacent cards- the only difference is in how those two cards were created. S^* also incurs a lower mana cost than the given optimal solution S . This creates a contradiction, as S was assumed to be optimal. Therefore, no S'' can be constructed, and S' must be an optimal solution for the subproblem $B[1...c]$.

A proof for the optimality of y 's subproblem follows from a substitution of variables.

7: Runtime

The p and m tables are both $\Theta(n)$ tall and $\Theta(n)$ wide. Each cell's computation takes $\Theta(n)$ time, making the total runtime $2 \times \Theta(n^3) = \Theta(n^3)$.

Presentation 2

The library is undergoing construction and the books on the shelves need to be moved. All of the books have the same height and width, but each book has a different thickness (in inches). The librarian has three identical boxes that can each fit at most m inches' worth of books. Given a list $P = (p_1, \dots, p_n)$ of thicknesses for each of the n books in the library, design an efficient algorithm to determine whether the librarian can fit all of the books into the boxes (each book must fit into one of the boxes and books cannot be split between two boxes).

Example

Input: $m = 4$, $P = \{2, 1, 4, 2, 1\}$

Output: True

Explanation: We need to fit books with thicknesses $\{2, 1, 4, 2, 1\}$ into three boxes, each with a capacity of 4 inches. A possible distribution of books could be:

- **Box 1:** 2 books 2 inches thick ($2 + 2 = 4$)
- **Box 2:** 1 book 4 inches thick (4)
- **Box 3:** 2 books 1 page thick ($1 + 1 = 2$)

Since all books fit into the boxes without exceeding any box's capacity, the result is **True**.

Input: $m = 4$, $P = \{3, 3, 3, 2\}$

Output: False

Explanation: We need to fit books with thicknesses $\{3, 3, 3, 2\}$ into three boxes, each with a capacity of 4 inches.

- There are 3 books that are each 3 inches thick.
- Two 3-inch books cannot fit in one box, so *one* 3-inch book must go in *each* box.
- With each box occupied by a 3-inch book, there is only 1 inch of space left in each.
- Therefore, the 2-inch thick book cannot fit in any of the boxes.

Even though the total amount of space available is 12 inches, and the total width of the books is less than 12, it is impossible to store all the books in three boxes. Therefore, the output is **False**

Presentation 2 Solution

If students used any of the two following greedy choices, then there exist counterexamples to prove sub-optimality.

- Fit the largest book in the box with the most open space

$$m = 10, P = [7, 5, 3, 2, 2, 2, 2, 2, 2, 2]$$

This places the books 7, 5, and 3 in each of the 3 boxes, leaving each able to fit 1, 2, and 3 of the 2-inch-thick books, respectively. That leaves a 2-inch book with nowhere to fit. However, the books can be fit by putting books 7 and 3 in one box, filling it completely, then storing all of the smaller books in the other boxes.

- Fit the largest book in the first box where it can fit

$$m = 20, P = [12, 11, 10, 7, 6, 5, 2, 2, 2, 2]$$

Books 12, 11, and 10 will obviously go in each of the 3 boxes, then 7, 6, and 5 will be added in order, to fill the boxes to a total of 19, 17, and 15 inches (respectively). No 2-inch book can fit in the first box, 1 can fit in the second, and 2 can fit in the third- that leaves a 2-inch book with nowhere to go. However, this set of books *can* fit in the boxes if Book 6 is placed alongside Book 12, Book 7 is placed alongside Book 11, and Book 5 is placed alongside Book 10. That leaves the boxes filled to 18, 18, and 15 inches, respectively. A 2-inch book can go in each of the first two boxes, and two 2-inch books can go in the second, accomodating all 4.

1: The Last Decision

The last decision we want to make, is whether the very last book can fit in any of the boxes, given the open space available after optimally filling each with the previous books.

2: Subproblem Definition

In this problem, we are given three identical boxes with size m inches. In addition, we are given a list, P , containing the the thickness in inches for each of the n books in the library. We could structure the recursive formula to represent whether the first i books fit, given that we have j, k , and l open space in each of the three boxes, which requires 4 dimensions in the table. We can, however, achieve a more tractable 3-dimensional table by choosing a recursive formula that represents whether the first i books can sum to j and k respectively (to be placed in boxes 1 and 2). We can use this strategy because if we know how much space is taken up in 2 of the boxes, and we know the number of books placed in all the boxes, we can determine how much space the Box 3 has used.

3: Recurrence Relation

$$B[i][j][k] = \begin{cases} \text{False} & \text{if } j < 1 \text{ or } k < 1 \\ \text{True} & \text{if } i, j, k = 1 \\ \text{False} & \text{if } i = 1, j, k > 1 \\ B[i-1][j-p_i][k] \vee & \\ B[i-1][j][k-p_i] \vee & \\ B[i-1][j][k] & \text{otherwise} \end{cases}$$

4: Pseudocode

Algorithm 3 Determining Whether Books Fit in Boxes

```

1: procedure books-fit?( $P, n, m$ )
2:    $dp \leftarrow$  3D matrix of dimensions  $((n + 1) \times (m + 1) \times (m + 1))$  init. to False
3:    $dp[1][1][1] \leftarrow \mathbf{True}$  ▷ Case 2
4:
5:   for  $i \leftarrow 2$  to  $n$  do
6:     for  $j \leftarrow 1$  to  $m$  do
7:       for  $k \leftarrow 1$  to  $m$  do
8:          $\text{box1} \leftarrow j - P[i]$ 
9:         if  $\text{box1} \geq 0$  then
10:           $dp[i][j][k] \leftarrow dp[i][j][k] \vee dp[i - 1][j - P[i]][k]$  ▷ Case 4
11:        end if
12:
13:         $\text{box2} \leftarrow k - P[i]$ 
14:        if  $\text{box2} \geq 0$  then
15:           $dp[i][j][k] \leftarrow dp[i][j][k] \vee dp[i - 1][j][k - P[i]]$  ▷ Case 4
16:        end if
17:      end for
18:    end for
19:  end for
20:
21:   $\text{totalSize} \leftarrow 0$  ▷ Check whether 3rd box overflows on True entries
22:   $\text{possible} \leftarrow \mathbf{False}$ 
23:  for  $i \leftarrow 2$  to  $n$  do
24:     $\text{totalSize} \leftarrow \text{totalSize} + P[i]$ 
25:    for  $j \leftarrow 1$  to  $m$  do
26:      for  $k \leftarrow 1$  to  $m$  do
27:        if  $B[i][j][k] = \mathbf{True}$  and  $\text{totalSize} - (m - j) - (m - k) \leq m$  then
28:           $\text{possible} \leftarrow \mathbf{True}$ 
29:        end if
30:      end for
31:    end for
32:  end for
33:  return  $\text{possible}$ 
34: end procedure

```

5: Example

$m = 4, P = \{2, 1, 4, 2\}$

We will begin by generating the table in accordance with the recursive formula:

$B[1][][] =$	j^k	1	2	3	4	5	$B[2][][] =$	j^k	1	2	3	4	5
	1	T	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>		1	T	T	<i>F</i>	<i>F</i>	<i>F</i>
	2	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>		2	T	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>
	3	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>		3	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>
	4	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>		4	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>
$B[3][][] =$	5	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	$B[4][][] =$	5	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>
	j^k	1	2	3	4	5		j^k	1	2	3	4	5
	1	T	T	<i>F</i>	<i>F</i>	T		1	T	T	T	T	T
	2	T	<i>F</i>	<i>F</i>	<i>F</i>	T		2	T	<i>F</i>	T	<i>F</i>	T
	3	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>		3	T	T	<i>F</i>	<i>F</i>	T
$B[5][][] =$	4	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>		4	T	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>
	5	T	T	<i>F</i>	<i>F</i>	<i>F</i>		5	T	T	T	<i>F</i>	T
	j^k	1	2	3	4	5		j^k	1	2	3	4	5
	1	T	T	T	T	T		1	T	T	T	T	T
	2	T	<i>F</i>	T	<i>F</i>	T		2	T	<i>F</i>	T	<i>F</i>	T
$B[5][][] =$	3	T	T	<i>F</i>	<i>F</i>	T	$B[5][][] =$	3	T	T	<i>F</i>	<i>F</i>	T
	4	T	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>		4	T	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>
	5	T	T	T	<i>F</i>	T		5	T	T	T	<i>F</i>	T
	j^k	1	2	3	4	5		j^k	1	2	3	4	5
	1	T	T	T	T	T		1	T	T	T	T	T

Then, when we reach $B[4][1][1]$, our total size will be 9, and $\text{totalSize} - (4-1) - (4-1) = 3$, which is $< m$, so we will set possible to **True**.

0.1 6: Proof of Optimal Substructure

This solution uses induction to prove optimal substructure, but students may also use exchange of arguments if it's sensible.

Base Case: If there are no books to place, then the books take up zero space. The only combination of nonnegative j and k s.t. $j + k = 0$ are $j, k = 0$. Therefore, we set $B[0][0][0] = \text{True}$, and all other entries $B[0][j][k] = \text{False}$.

Inductive Step: Assume that we have n books to place, and the remaining space left in box 1 and 2, i and j , is nonnegative. Also, assume that the entries for placing $n - 1$ books have been calculated correctly for all possible configurations of open space in box 1 and 2.

There are only 3 boxes to place book n into. Each of the three inductive cases considers taking away $P[n]$ inches of space from each box, i.e. remaining space ($j - p[n]$, $k - p[n]$, or $l - p[n]$). We don't explicitly take away space from Box 3, and instead leave the first two boxes the same and calculate whether the total space left, $(\sum_{i=1}^n p_i) - (m - j) - (m - k)$, is less than or equal to m . If at least one of these three situations results in no overflow, then we assign the result of this situation to the result of trying to place $n - 1$ books optimally for all space configurations. Since these subproblems are correct and we take the logical OR of these three possibilities, then the choice we make for the n books given the original values of space remaining (i and j) is correct.

Runtime Analysis

There are nm^2 total table entries, and each entry takes a constant amount of time to calculate. After the table is loaded completely, we do one more sweep through the table to determine if entries labeled **True** have enough space in the 3rd box. Since we use a cumulative counter to keep track of the sum of pages, this conditional check occurs in constant time at each step. Therefore, this sweep takes nm^2 time. The complete runtime of the algorithm is the sum of these two operations.

$$T(n, m) = nm^2 + nm^2 = \Theta(nm^2)$$